

Distributed Sparse Matrix–Vector Multiplication (SpMV) with MPI CUDA-aware

Matilde Munaretto

GitHub repository: https://github.com/matilde03-research/GPU_Computing

Master’s Degree in Artificial Intelligence Systems , DISI, UniTN

matilde.munaretto@studenti.unitn.it matricola: 266680

Abstract—This investigation was mainly about implementing and evaluating a distributed-memory sparse matrix-vector multiplication (SpMV) across up to four NVIDIA A30 GPUs, using 1D cyclic (modulo) row partitioning and CUDA-aware MPI. Two local kernels (a hand-written CSR-vector kernel and NVIDIA cuSPARSE) are validated against an OpenMP CPU reference. Across ten SuiteSparse matrices and synthetic R-MAT and 3D-stencil generators, a single A30 reaches up to ~ 200 GFLOP/s, but multi-GPU execution is heavily *communication-bound*: speedups remain below one (best ~ 0.20 at $P=2$ for the 71.7M-nonzero bone010), because the halo exchange dominates the sub-millisecond local compute. This study characterizes when 1D partitioning balances load, when the communication bounds performance, and the problem-size limit imposed by a rank-0 read-and-distribute model.

I. INTRODUCTION

Sparse matrix–vector multiplication, $y = Ax$, is a fundamental kernel of scientific computing: it is the propagation step of graph analytics (e.g. PageRank), and a building block of many machine-learning workloads. SpMV is strongly *memory-bound*: each nonzero contributes only one multiply-add but requires irregular, data-dependent access to the input, so performance is governed by memory and access locality rather than by raw compute, SpMV is not a math-bound problem.

A single GPU is limited by its memory *capacity* (the matrix must fit) and its memory *bandwidth*. Distributing SpMV over several GPUs promises both more aggregate capacity and more aggregate bandwidth, but the same low arithmetic intensity that makes SpMV memory-bound also makes it particularly sensitive to inter-GPU communication: every distributed multiply needs remote elements of x .

In this section of the report, it is also appropriate to outline the high-level methodology followed to frame the issue addressed: Foster’s Design Methodology. It’s composed as follows:

- 1) Partitioning (domain decomposition): divide the data (e.g. matrix blocks, grid cells) into subdomains. Static partitioning is used in this study.
- 2) Communication: determine required message patterns
- 3) Agglomeration: combine tasks and optimize message overhead
- 4) Mapping: assign tasks to processing elements (GPUs)

This work asks a practical question: *under the simplest standard distribution (1D cyclic row partitioning) and CUDA-aware MPI, when does multi-GPU SpMV actually pay off*

on a cluster of A30 GPUs? This work is ment to delve into strong and weak scaling across real and synthetic matrices, the balance between computation and communication, and the problem-size limits of the approach. The short answer, supported throughout, is that for realistic matrix sizes on this hardware and **this software setting** (1D cyclic partitioning and CRS-Vector as computation kernel) a single GPU is already the better choice. Further explanations will be discussed.

II. METHODOLOGY

Formats and kernels. Rank 0 reads each matrix in Matrix Market coordinate (COO) form. Then the same rank distributes matrix entries to all processes. Every rank then converts its assigned entries to compressed sparse row (CSR). Two local SpMV kernels are evaluated: (I) a hand-written CSR-vector kernel (one warp per local row, warp-shuffle reduction), and (II) NVIDIA cuSPARSE generic SpMV. Both use single precision (float32).

Partitioning. It was used 1D modulo (cyclic) row partitioning with row ownership rule $\text{owner}(i) = i \bmod P$, where P is the number of GPUs. Each rank stores all nonzeros of its rows, the local row pointer is indexed by the local index i/P while column indices remain global. The dense vector x is distributed by the same rule (x_j owned by $j \bmod P$). Each rank identifies the distinct remote (“ghost”) columns it references and fetches them through a point-to-point halo exchange (non-blocking MPI_Isend/Irecv to the actual neighbours), which after some preliminary trials was considered to be faster than a CUDA-aware MPI_Alltoallv on this stack. An optional owned/ghost column-split allows the owned-column partial product to overlap the exchange.

Validation. For every run, it’s computed a reference result with an OpenMP CSR multiply using the same operand vector; the distributed result is gathered, reordered from the cyclic layout, and compared. In the extensive results reporting present on the GitHub repository related to this report there are also details about the relative L_2 error and the maximum absolute error; all runs reported here pass ($L_2\text{-rel} < 10^{-3}$, typically 10^{-7} – 10^{-9}).

Strong scaling and Weak scaling. Strong scaling measures how the execution time of a fixed-size problem decreases as the number of processors increases. Ideally, doubling the number of processors halves the runtime. Weak scaling measures

how the execution time changes when the problem size grows proportionally with the number of processors, keeping the amount of work per processor constant. Ideally, the runtime remains constant as both the problem size and processor count increase.

Measurement. Each configuration runs 4 warm-up and 100 timed iterations. Timed regions are bracketed by `MPI_Barrier` and reduced as the maximum over ranks (the slowest rank gates the collective). The results include the per-SpMV wall time, the kernel-only *computation* time, the *communication* time (wall minus computation), the FLOP count ($2 \cdot nnz$), the speedup T_1/T_P (where T_1 is a single-GPU baseline measured in the same run), the efficiency $T_1/(PT_P)$, the nonzero load balance per rank, the ghost (communication) volume, and the device memory footprint per rank. The random operand dense vector of compatible size is generated with a fixed seed (42) for reproducibility. Float32 was used as a data type.

Environment. Experiments run on a university cluster node with four NVIDIA A30 GPUs; one MPI rank is bound to one GPU. Software: CUDA 12.5, OpenMPI 4.1.5 (CUDA-aware), GCC 12.3; built with `nvcc -arch=sm_80` and `-fopenmp`.

III. DATASET

The dataset is composed of the 10 sparse matrices downloaded from SuiteSparse for the previous deliverable. For matrices details and characterization please refer to the first report. These matrices are used for strong scaling. This time we also introduced 4 randomly generated matrices: 2 from R-MAT and 2 from Structured stencil. R-MAT are power-law (skewed), while Structured stencil are constant (5/7/27 nnz/row), these matrices are used for weak scaling.

IV. RESULTS

In this section results are shown in Table I and Table II and further discussed in the following sections.

V. DISCUSSION

Table I summarises strong scaling (cuSPARSE) on ten SuiteSparse matrices spanning 1.5–71.7M nonzeros and 3–73 nonzeros/row, and Table II gives the complete per-rank CSR-vector breakdown (time, speedup, efficiency, FLOPs, memory, nonzero balance, and communication volume). The cuSPARSE case is actually a reference for non-overlapped communication, while CSR-vector attempts overlap. So cuSPARSE shows what happens without overlap, it’s a control for that specific technique. Figure 1 shows the computation/communication split of the hand-written CSR-vector kernel for all ten matrices at 1, 2 and 4 GPUs, and Fig. 2 the weak-scaling behaviour of the two synthetic generators. Algorithm 1 gives the per-iteration distributed step that every measurement exercises: the local matrix is split by column owner so that the owned-column product can be computed *while* the point-to-point halo for the ghost columns is in flight.

Algorithm 1 Distributed SpMV step on rank r with computation/communication overlap (one iteration of $y \leftarrow Ax$ under 1D cyclic partitioning).

Require: rank r owns rows $\{i : i \bmod P = r\}$; local CSR split by column owner into A_{own} (cols j with $j \bmod P = r$) and A_{ghost} ; owned slice x_{loc} ; global buffer x (owned entries prefilled)

- 1: pack $s[m] \leftarrow x_{\text{loc}}[\text{send_local}[m]]$ $\triangleright$ owned values neighbours need
- 2: **for** each p with $\text{recv_cnt}[p] > 0$ **do**
- 3: post `MPI_Irecv` of ghosts from rank p
- 4: **end for**
- 5: **for** each p with $\text{send_cnt}[p] > 0$ **do**
- 6: post `MPI_Isend` of s to rank p
- 7: **end for**
- 8: $y \leftarrow A_{\text{own}} x$ $\triangleright$ owned-col. SpMV *overlaps* the halo
- 9: `MPI_Waitall` $\triangleright$ exposed communication
- 10: $x[\text{recv_global}[k]] \leftarrow \text{recvbuf}[k]$ $\triangleright$ scatter ghosts into x
- 11: $y \leftarrow y + A_{\text{ghost}} x$ $\triangleright$ ghost-col. SpMV (accumulate)

A. When 1D cyclic partitioning works well

Cyclic partitioning assigns the elements in a “round-robin” fashion [1]. It balances the *number of rows*, which balances *work* only when row degrees are uniform. This holds for regularly structured matrices: the 3D-stencil generator and the structural meshes (ldoor, bone010) show near-perfect nonzero balance (min $\approx$ avg $\approx$ max per rank). It fails for power-law matrices because cyclic ownership inherits R-MAT’s skewed bit distribution (balancing row count does not balance nonzeros). The other regime in which the approach “works” is *single-GPU*: cuSPARSE reaches 80–203 GFLOP/s and the CSR-vector kernel 50–115 GFLOP/s on a single A30, with near-ideal efficiency at $P=1$. In the multi-GPU regime, the only matrices that come close to amortising the halo are the largest and densest: bone010 (72M nnz) reaches the best observed efficiency, 0.10 at $P=2$ (Table I). The trend in the S_2 column is monotone in size, from 0.011 (boyd2) to 0.201 (bone010), confirming that larger matrices are progressively less penalised, even though none seems to benefit from multi-GPU here.

B. When the implementation is interconnect-bound

At $P \geq 2$ the implementation is interconnect-bound essentially always. Figure 1 makes this concrete: **the local computation shrinks** with P (work per GPU falls), while the communication *grows* and reaches 90–99% of the wall time. Quantitatively, halos of 0.7–3 MB per rank take 2–17 ms, an effective bandwidth of only ~ 0.05 – 0.5 GB/s (far below PCIe or NVLink). The root cause is dual. First, cyclic partitioning destroys locality: a rank’s scattered rows reference columns owned by every other rank, so each rank needs $\sim (P-1)/P$ of x , effectively an all-to-all. Cyclic partitioning improves load balance at the expense of locality [2]. Second, SpMV’s low intensity leaves almost nothing to hide: cuSPARSE compute

TABLE I
STRONG SCALING (CUSPARSE). G_1 : SINGLE-GPU THROUGHPUT. $S_P = T_1/T_P$: SPEEDUP OVER ONE GPU; $E_P = S_P/P$: PER-RANK EFFICIENCY.
 $S_P < 1$ (HENCE $E_P \ll 1$) MEANS MULTI-GPU IS SLOWER THAN ONE GPU.

Matrix	nnz (M)	nnz/row	G_1 (GF/s)	S_2	E_2	S_4	E_4
boyd2	1.5	3.2	80.6	0.011	0.006	0.004	0.001
ASIC	2.3	3.4	80.6	0.021	0.010	0.008	0.002
webbase	3.1	3.1	96.5	0.027	0.014	0.005	0.001
Ruccil	7.8	3.9	130.8	0.053	0.027	0.024	0.006
Ga41	18.5	69.0	169.8	0.064	0.032	0.032	0.008
eu-2005	19.2	22.3	168.7	0.078	0.039	0.030	0.008
rajat31	20.3	4.3	124.6	0.049	0.025	0.031	0.008
FullChip	26.6	8.9	145.3	0.070	0.035	0.031	0.008
ldoor	46.5	48.9	193.4	0.154	0.077	0.062	0.016
bone010	71.7	72.6	203.4	0.201	0.101	0.081	0.020

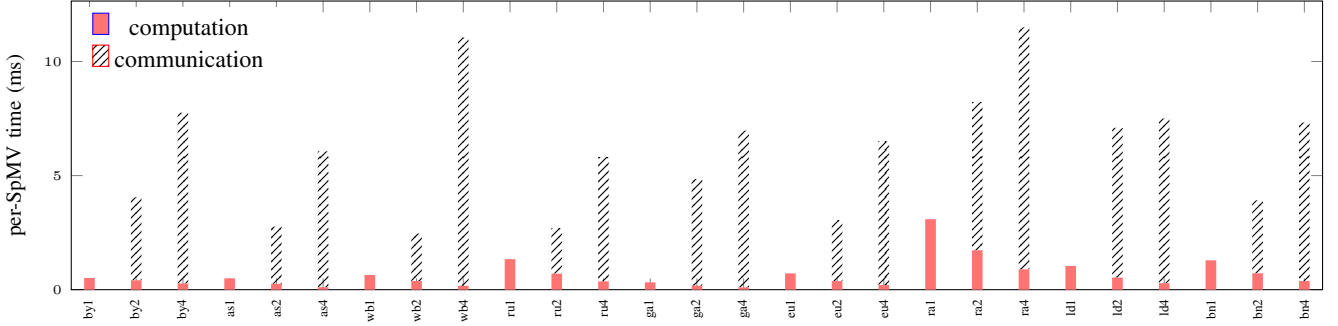


Fig. 1. Computation vs. communication (CSR-vector) for all ten matrices at $\{1, 2, 4\}$ GPUs (label suffix = number of GPUs; by=boyd2, as=ASIC, wb=webbase-1M, ru=Ruccil, ga=Ga41, eu=eu-2005, ra=rajat31, ld=ldoor, bn=bone010). At $P=1$ there is obviously no halo; at $P \geq 2$ communication dominates the shrinking computation.

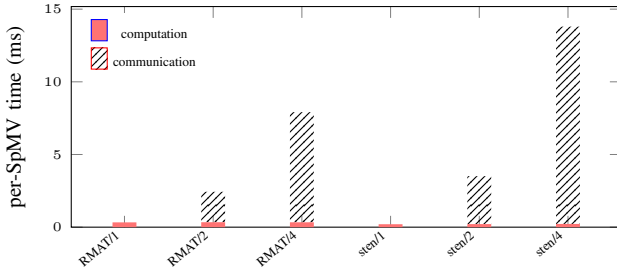


Fig. 2. Weak scaling (CSR-vector): per-GPU work is held constant as P grows, for the power-law R-MAT generator and the 3D 7-point stencil. Ideal weak scaling is flat; instead the wall time grows because communication scales with P .

is 0.04–0.7 ms. A simple model, multi-GPU wins when $T_1 > \frac{P}{P-1} T_{\text{comm}}$, places the crossover at tens of millions of nonzeros for the hand kernel; only bone010 approaches it. *Weak scaling* (Fig. 2) tells the same story from the other side: with per-GPU work held constant, the wall time should stay flat but instead grows 0.22 to 2.3 to 7.7 ms (R-MAT) and 0.20 to 3.5 to 13.8 ms (stencil), because the communication grows linearly with P . We also note that the *overlap* in Algorithm 1 (the owned-column product $A_{\text{own}}x$ computed while the halo is in flight) did not help on this study: CUDA-aware intra-node transfers share the GPU with the compute kernel (contention rather than hiding), and the cyclic layout leaves only $\sim 1/P$

of an already-tiny compute to overlap.

C. Largest matrix the approach can manage

The binding limit in this setting is not GPU memory but the rank-0 *read-and-distribute* model. Rank 0 holds the entire matrix simultaneously, the full COO, the bucketed send buffers, and the full reference CSR, roughly 30–40 bytes per nonzero of host memory, and the serial COO→CSR build dominates set-up (e.g. ~ 17 s for bone010, seconds for ldoor and FullChip, this can be seen in the extended results). We successfully processed up to 71.7 M nonzeros (bone010. Per-GPU *device* memory is comfortably below the limit: even with the $O(N)$ full- x replication and the local CSR, footprints stay at 17–145 MB/rank. In this setting a parallel/distributed reader (each rank reading its own slice) could be an help to remove the rank-0 bottleneck.

VI. CONCLUSION

Lessons learned. (1) In our multi-GPU case 1D cyclic SpMV is communication-bound for realistic matrix sizes; a single A30 with cuSPARSE already solves these problems and is the faster choice up to $\sim 10^7$ – 10^8 nonzeros. (2) Cyclic partitioning balances *rows*, hence *work* only for uniform-degree matrices; power-law matrices (R-MAT) are imbalanced and would need nonzero-aware partitioning. (3) The communication *transport* matters: point-to-point messaging was faster than CUDA-aware Alltoallv on this stack, whereas

TABLE II

FULL PER-MATRIX CSR-VECTOR RESULTS FOR $P \in \{1, 2, 4\}$ GPUS. T : WALL TIME PER SpMV; $S_P = T_1/T_P$ AND $E_P = S_P/P$ ARE SPEEDUP AND PER-RANK EFFICIENCY (ABSENT AT $P=1$); FLOPS = $2 \cdot nnz$; NNZ/RANK AND MEMORY ARE PER RANK; "COMM" IS THE HALO VOLUME OF THE BUSIEST RANK.

Matrix	FLOPs	P	T [ms]	S_P	E_P	nnz/rank [M]			Mem [MB]	Comm [MB]
						min	avg	max		
boyd2	3.0×10^6	1	0.51	–	–		1.50		18.56	–
		2	4.04	0.148	0.074	0.74	0.75	0.76	13.73	0.89
		4	7.56	0.079	0.020	0.33	0.38	0.43	8.78	0.81
ASIC	4.7×10^6	1	0.50	–	–		2.33		28.19	–
		2	2.76	0.178	0.089	1.16	1.16	1.17	18.38	0.75
		4	6.06	0.082	0.020	0.58	0.58	0.58	11.81	0.70
webbase	6.2×10^6	1	0.64	–	–		3.11		38.95	–
		2	2.46	0.294	0.147	1.54	1.55	1.56	21.97	0.15
		4	11.05	0.065	0.016	0.76	0.78	0.78	13.39	0.20
Rucci1	1.6×10^7	1	1.34	–	–		7.79		75.37	–
		2	2.70	0.464	0.232	3.90	3.90	3.90	38.73	0.21
		4	5.81	0.224	0.056	1.95	1.95	1.95	20.41	0.31
Ga41	3.7×10^7	1	0.32	–	–		18.49		145.2	–
		2	4.85	0.065	0.032	9.24	9.24	9.25	75.1	0.51
		4	6.97	0.045	0.011	4.61	4.62	4.63	40.1	0.77
eu-2005	3.8×10^7	1	0.71	–	–		19.24		159.9	–
		2	3.05	0.232	0.116	9.61	9.62	9.63	87.2	1.40
		4	6.51	0.109	0.027	4.80	4.81	4.82	49.0	1.65
rajat31	4.1×10^7	1	3.09	–	–		20.32		226.6	–
		2	8.22	0.350	0.175	10.16	10.16	10.16	152.1	7.46
		4	11.50	0.249	0.062	5.08	5.08	5.08	99.5	7.69
FullChip	5.3×10^7	1	10.05	–	–		26.62		248.7	–
		2	14.66	0.659	0.330	13.15	13.31	13.47	152.7	5.67
		4	17.75	0.574	0.143	5.52	6.66	7.85	99.2	7.90
ldoor	9.3×10^7	1	1.05	–	–		46.52		369.5	–
		2	7.09	0.139	0.069	23.26	23.26	23.27	193.8	1.82
		4	7.50	0.133	0.033	11.63	11.63	11.63	106.0	2.72
bone010	1.4×10^8	1	1.29	–	–		71.67		561.8	–
		2	3.91	0.345	0.173	35.83	35.83	35.83	290.3	1.88
		4	7.32	0.186	0.046	17.91	17.92	17.92	154.6	2.82

computation/communication *overlap* was neutral-to-negative because intra-node CUDA-aware transfers contend with the compute kernel. (4) The rank-0 read-distribute model, not device memory, caps the problem size. The implementation correctly realizes the Foster design methodology: clean separation of partitioning (1D cyclic), communication (point-to-point to real neighbours), agglomeration (non-blocking overlap), and mapping (one rank per GPU). The code handles all matrix types (10 SuiteSparse, R-MAT, stencil) correctly, validated against OpenMP CPU. Furthermore it was given a chance to a simple computation/communication overlap mechanism that is properly implemented (even though it doesn't seem to pay off). Distributed memory programming with MPI is performed and all needed MPI collectives (Scatterv, Alltoall, Alltoallv, Gatherv, Reduce, Barrier) are correctly implemented and verified.

Practical takeaways. Multi-GPU SpMV pays off for very large, locality-friendly matrices, and when there's an efficient managing of computation/communication overlap. For the matrices and software produced for this deliverable, the right choice is a single GPU. Some improvements for making multi-GPU worthwhile are: a locality-preserving partition (1D block or graph/hypergraph) [1] to cut the

halo from $O(N)$ toward surface-to-volume; nonzero-aware balancing for skewed matrices; parallel I/O to lift the size cap.

Note: complete results for every matrix, every combination of 1/2/4 GPU(s), and validation results can be found under the directory "CompleteResults/1D" in the GitHub repository prepared for this deliverable

Disclaimer: In the last week (from 22/06/2026 to 28/06/2026) before this report release the Baldo DISI cluster was particularly crowded and this was a bit of an obstacle to run experiments and investigate further techniques to improve the computation/communication overlap.

REFERENCES

- [1] Pacheco *An Introduction to Parallel Programming*, 2021, second edition.
- [2] UC San Diego, *Lecture 7 (1/29/02): Partitioning and Load Balancing*. [Online]. Available: https://cseweb.ucsd.edu/~baden/classes/cse262_wi02/lectures/Lec07/Lec07.html. [Accessed: 26/06/2026].